

Justificaciones de Diseño y Arquitectura

Casos de Uso

- **Iniciar Sesión:** Decidimos no incluir "Iniciar Sesión" en el diagrama general para evitar sobrecarga visual y mantener el gráfico enfocado en el negocio. Al ser un comportamiento transversal, lo establecimos como una precondition obligatoria en la especificación de cada caso de uso.
- **Registrar Donante y Entidad Beneficiaria:** Optamos por separarlos en dos casos de uso distintos para mantener una **Alta Cohesión Funcional**. Los datos y reglas de validación son muy diferentes: a los donantes les validamos datos identitarios (DNI, CUIT), mientras que de las entidades nos interesan sus datos institucionales, necesidades y representantes.
- **Exponer Notificación:** Aislamos la mensajería en este caso de uso, que es incluido (<<include>>) por la importación y los registros. Lo conectamos con el actor secundario "Servicio Externo de Notificación" para representar la integración con plataformas como Email, SMS y WhatsApp sin acoplar el dominio a tecnologías específicas.

Modelo de Dominio

- **Representante:** Modelamos al representante como una entidad independiente. Al ser un contacto administrativo de una **EntidadBeneficiaria**, obligarlo a heredar de una clase base de "Usuario" o "Donante" mezclaría responsabilidades y violaría el **Principio de Responsabilidad Única (SRP)**.
- **Donante (Persona Humana y Jurídica):** Aplicamos el **Principio de Composición sobre Herencia**. En lugar de usar herencia clásica, la entidad **Donante** compone a las personas. Esto nos permite centralizar la cuenta y el historial de donaciones en un solo lugar, evitando la duplicación de datos y simplificando el modelo relacional de la base de datos.
- **Contacto Predeterminado:** **Donante** tiene una lista de **MedioContacto**, pero agregamos una referencia directa al contacto elegido por el usuario como predeterminado. Esto respeta el **Encapsulamiento** y evita iterar la lista al momento de notificar, reduciendo la complejidad computacional a **O(1)**.
- **Bienes y Categorías:** Unificamos todo en una sola clase **Bien**. Usamos el **Patrón Type Object** delegando en **Categoria** la responsabilidad de dictar qué atributos (flags booleanos) son obligatorios. Elegimos este camino para evitar la **explosión combinatoria de clases** (por ejemplo, tener que crear subclases como **BienPerecederoConEstado**).
- **Segmentación de Donaciones:** Creamos **RegistroDonacion** para representar el ingreso de mercadería al depósito. Funciona aplicando el **Patrón Factory**: recibe bultos mezclados y los divide automáticamente en objetos **Donacion** individuales agrupados por subcategoría, protegiendo así la consistencia del stock.
- **Estado de la Donación:** Utilizamos el **Patrón Historial de Estados (Audit Log)** con la clase **CambioEstado**. En lugar de sobrescribir un atributo de estado simple,

guardamos cada movimiento con su respectiva fecha. Esto asegura la trazabilidad total del ciclo de vida de la donación sin perder información pasada.

- **Desacoplamiento de Logística:** Basándonos en **Domain-Driven Design (DDD)**, tratamos a la logística como un Contexto de Soporte. Clases como **Ruta** o **Camion** desconocen las reglas sociales de la ONG; solo procesan secuencias de coordenadas geográficas. Esto nos da alta cohesión técnica y separa bien las responsabilidades.

Arquitectura e Integración

- **División en Microservicios:** Dividimos el sistema en tres microservicios independientes (**donaciones**, **logística** y **notificaciones**) más una librería común (**common-lib**), comunicados vía REST HTTP. Esto busca maximizar la cohesión y minimizar el acoplamiento. Donaciones centraliza el negocio, Logística resuelve el transporte y Notificaciones aísla el uso de APIs externas. Al separarlos así logramos tolerancia a fallos: si se cae el proveedor de mensajería o el de mapas, el módulo principal de donaciones sigue funcionando.
- **Patrón DTO:** En todos los controladores implementamos el **Patrón DTO** usando clases **Request** y **Response**. Esto evita exponer nuestras entidades internas y previene vulnerabilidades de seguridad como la Asignación Masiva (*Mass Assignment*).
- **Asignación de Donaciones:** Elegimos el **Patrón Strategy** para el motor de matchmaking. Nos permite intercambiar algoritmos de asignación dinámicamente y cumplir con el **Principio Abierto/Cerrado (OCP)**, facilitando agregar nuevos criterios en el futuro sin modificar el código base.
- **Tareas Programadas (Schedulers):** Usamos **@Scheduled** para automatizar procesos recurrentes o pesados, como el motor de matchmaking o el armado de lotes para envíos logísticos. Al correr estas tareas en *background*, evitamos bloquear al usuario.
- **Eventos y Notificaciones (Patrón Observer):** Aplicamos el **Patrón Observer** para enviar alertas. Cuando pasa algo (ej. una entrega exitosa), se emite un evento y los *Listeners* lo escuchan para llamar al microservicio de notificaciones por REST. Para no trabar el sistema, ejecutamos esta llamada asincrónicamente usando **CompletableFuture.runAsync()**. Por su parte, el envío final usa el **Patrón Strategy** para elegir dinámicamente entre Email, SMS o WhatsApp.
- **Planificación de Rutas (Adapter y Webhooks):** Usamos el **Patrón Adapter** para separar nuestro código del formato de la API externa de ruteo. Como calcular rutas es un proceso lento, diseñamos un **Webhook (Callback)**: nuestro sistema manda las coordenadas en lotes de hasta 100 (respetando los límites de la API externa), libera la conexión, y recibe la respuesta tiempo después en un endpoint aparte. Lo que no se pudo planificar en ese lote, el sistema lo re-encola automáticamente.